

INTRODUCCIÓN A LA PROGRAMACIÓN

Guía de Repaso N°2

Funciones en Python

for · while · return · parámetros · funciones encadenadas

Nombre: _____ Fecha: _____
Curso: _____ Nota: _____ / 100

 Instrucciones

Esta guía tiene 8 ejercicios de programación en Python sobre funciones.
Lee cada enunciado con atención antes de escribir código.
Puedes usar cualquier hoja borrador, pero la respuesta final va en las líneas de código.
Cada ejercicio indica el puntaje y los temas que evalúa.
Las funciones deben usar def, parámetros y return salvo que el enunciado indique lo contrario.

#	Ejercicio	Temas	Pts
1	Temperatura corporal	función + if/elif/else	10
2	Tabla de multiplicar	función + for	10
3	Contar vocales	función + for + str	10
4	Serie de Fibonacci	función + while	10
5	Número perfecto	función booleana + for	15
6	Cifrar con César	función + for + str	15
7	Estadísticas de notas	funciones encadenadas	15
8	Validar RUT (simplificado)	función + while + str	15

Ejercicio 1 Temperatura corporal

★ Básico

10 pts

 Enunciado

Escribe una función `clasificar_temperatura(temp)` que reciba un número decimal y retorne:

- $temp < 36.0 \rightarrow$ "Hipotermia"
- $36.0 \leq temp \leq 37.5 \rightarrow$ "Normal"

- $37.5 < \text{temp} \leq 39.0 \rightarrow \text{"Fiebre"}$
- $\text{temp} > 39.0 \rightarrow \text{"Fiebre alta"}$

Luego imprime la clasificación para: 35.5, 36.8, 38.1, 40.2

Tu código Python:

```
# Tu código aquí:
```



35.5 °C → Hipotermia
36.8 °C → Normal
38.1 °C → Fiebre
40.2 °C → Fiebre alta

Ejercicio 2 Tabla de multiplicar

★ Básico

10 pts



Enunciado

Escribe una función `tabla(numero, limite=10)` que imprima la tabla de multiplicar de `numero` desde 1 hasta `limite` (inclusive).

Debe usar un ciclo `for`.

Llama a la función para imprimir la tabla del 6 completa y la tabla del 4 solo hasta el 5.

Tu código Python:

```
# Tu código aquí:
```

[illegible]

Tabla del 4 hasta 5: $4 \times 1 = 4 \dots 4 \times 5 = 20$

Ejercicio 3 Contar vocales

★ Básico

10 pts



Luego escribe otra función `porcentaje_vocales(texto)` que retorne qué porcentaje del texto son vocales (redondeado a 1 decimal).

Prueba ambas con: "Hola Mundo", "Python es genial", "AEIOU"

Tu código Python:

```
# Tu código aquí:
```



"Hola Mundo" → 4 vocales, 44.4%
"Python es genial" → 6 vocales, 37.5%
"AEIOU" → 5 vocales, 100.0%

Ejercicio 4 Serie de Fibonacci

★★ Intermedio

10 pts



Escribe una función `fibonacci(n)` que use un ciclo `while` para generar e imprimir los primeros `n` números de la serie de Fibonacci.

Recuerda: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Cada par de términos consecutivos se suman para obtener el siguiente.

Llama a la función con `n=8` y `n=12`.

Tu código Python:

```
# Tu código aquí:
```



`fibonacci(8)`: 0 1 1 2 3 5 8 13
`fibonacci(12)`: 0 1 1 2 3 5 8 13 21 34 55 89

Ejercicio 5 Número perfecto

★★ Intermedio

15 pts



Un número es perfecto si la suma de sus divisores propios (todos los divisores excepto él mismo) es igual al número.

Ejemplo: 6 es perfecto porque sus divisores son 1, 2, 3 y $1+2+3 = 6$.

Escribe:

1. `suma_divisores(n)` → retorna la suma de los divisores propios de `n`.

2. `es_perfecto(n)` → retorna `True` si `n` es perfecto, `False` si no.

Luego encuentra e imprime todos los números perfectos entre 1 y 1000.

Tu código Python:

```
# Tu código aquí:
```



6 es perfecto

28 es perfecto

496 es perfecto

Ejercicio 6 Cifrado César

★★ Intermedio

15 pts



El cifrado César desplaza cada letra del alfabeto una cantidad fija de posiciones.

Ejemplo con desplazamiento 3: A→D, B→E, Z→C (vuelve al inicio).

Escribe:

1. cifrar(texto, desplazamiento) → retorna el texto cifrado (solo desplaza letras, respeta mayúsculas/minúsculas, deja otros caracteres igual).
2. descifrar(texto, desplazamiento) → retorna el texto original (usa cifrar con desplazamiento negativo).

Prueba con: cifrar("Hola Mundo", 3) y descifra el resultado.

Pista: usa ord() y chr() para convertir entre caracteres y números.

Pista de sintaxis:

```
ord('A')          # → 65   (código ASCII)
chr(65)           # → 'A'  (carácter desde código)
# Para rotar dentro del alfabeto:
# nueva_pos = (pos_original - base + desplazamiento) % 26 + base
# donde base = ord('A') para mayúsculas, ord('a') para minúsculas
```

Tu código Python:

```
# Tu código aquí:
```



```
cifrar("Hola Mundo", 3) → "Krod Pxqgr"
descifrar("Krod Pxqgr", 3) → "Hola Mundo"
```

Ejercicio 7 Estadísticas de notas

★★★ Avanzado

15 pts



Escribe un sistema de análisis de notas con estas funciones:

1. promedio(notas) → retorna el promedio redondeado a 2 decimales. No uses sum().
2. maximo(notas) → retorna la nota más alta. No uses max().
3. minimo(notas) → retorna la nota más baja. No uses min().
4. sobre_promedio(notas) → retorna una lista con las notas que están sobre el promedio.
5. reporte(notas) → llama a todas las anteriores e imprime un informe completo.

Prueba con: [45, 72, 88, 55, 91, 63, 77, 49, 84, 60]

Tu código Python:

```
# Tu código aquí:
```



—— Reporte de Notas ——

Notas: [45, 72, 88, 55, 91, 63, 77, 49, 84, 60]
Promedio: 68.4
Máxima: 91
Mínima: 45
Sobre prom: [72, 88, 91, 77, 84]

Ejercicio 8 Validar RUT (simplificado)

★★★ Avanzado

15 pts



Escribe un sistema de validación de RUT chileno simplificado con estas funciones:

1. `limpiar_rut(rut_str)` → recibe un string como "12.345.678-9" y retorna solo los dígitos y el dígito verificador, sin puntos ni guión: "123456789".
 2. `calcular_dv(numero)` → recibe el número del RUT como entero (sin DV) y calcula el dígito verificador según el módulo 11:
 - Multiplica cada dígito por la secuencia 2,3,4,5,6,7 (repite si es necesario, de derecha a izquierda).
 - Suma los productos. Resta: $11 - (\text{suma} \% 11)$.
 - Si el resultado es 11 → '0'; si es 10 → 'K'; si no → el número como string.
 3. `validar_rut(rut_str)` → retorna True si el DV calculado coincide con el del string.
- Prueba con: "12.345.678-9", "11.111.111-1", "7.654.321-K"

Esquema del algoritmo módulo 11:

```
// Ejemplo: RUT 12345678
// Dígitos de derecha a izq: 8 7 6 5 4 3 2 1
// Multiplicadores:      2 3 4 5 6 7 2 3 (repite 2-7)
// Productos:            16+21+24+25+24+21+4+3 = 138
// 11 - (138 % 11) = 11 - 6 = 5 → DV = '5'... pero el DV es '9'
// (Nota: usa el número completo 12345678 en tu función)
```

Tu código Python:

```
# Tu código aquí:
```




12.345.678-9 → True
11.111.111-1 → True (o False, depende del DV real calculado)
7.654.321-K → True

REFERENCIA RÁPIDA

Funciones

```
def nombre(p1, p2=default):  
    # cuerpo  
    return valor  
  
resultado = nombre(a, b)
```

Ciclo while

```
while condicion:  
    ...  
    # actualizar condicion
```

Ciclo for

```
for i in range(inicio, fin+1):  
    ...  
for c in texto:  
    ...  
for item in lista:  
    ...
```

Strings útiles

```
len(s)  
s.lower() s.upper()  
s.isalpha() s.isdigit()  
ord(c) chr(n)  
'aeiou'.count(c)
```